

MySQL / MariaDB

Troubleshooting Runbook

Data Corruption & Deletion Recovery | Redo Log Troubleshooting

Comprehensive Step-by-Step Operational Guide

Document Type	Operational Runbook
Scope	MySQL 5.7+ / 8.x & MariaDB 10.x+
Audience	DBA / SRE / Infrastructure Engineers
Classification	Internal — Restricted
Version	3.0

Environment Filesystem Layout

Purpose	Path
Data Directory	/apps/mysql/data
Binary Logs	/apps/mysql/binlog
Backups	/apps/mysql/backups
Error / Server Log	/var/log/mysqld.log

Table of Contents

- **Section 1: Issue Identification & Diagnosis**

- 1.1 Detecting Data Corruption
- 1.2 Identifying Deleted Data via Binary Logs
- 1.3 Diagnosing Missing Redo Logs

- **Section 2: Precautionary Steps**

- 2.1 Backup Current State
- 2.2 Environment Safeguards
- 2.3 Communication & Change Management

- **Section 3: Remediation Steps**

- 3.1 Recovering Corrupted Tables
- 3.2 Recovering Deleted Data — mysqlbinlog Scenarios
 - Scenario 1: Recover Accidentally Deleted Rows (DELETE)
 - Scenario 2: Recover a Dropped Table (DROP TABLE)
 - Scenario 3: Recover from TRUNCATE TABLE
 - Scenario 4: Recover from Accidental UPDATE (wrong WHERE)
 - Scenario 5: PITR Skipping a Bad Transaction (GTID)
 - Scenario 6: Recover from a Dropped Database (DROP DATABASE)
 - Scenario 7: Selective Table Recovery from Full Binlog
- 3.3 Recovery via Mariabackup / Xtrabackup
- 3.4 Rebuilding Missing Redo Logs

- **Section 4: Post-Remediation Confirmation**

- 4.1 Data Integrity Verification
- 4.2 Performance & Stability Checks
- 4.3 Final Sign-Off Checklist

Section 1: Issue Identification & Diagnosis

The first and most critical phase is accurately diagnosing the nature and scope of the problem. Misdiagnosis can lead to worsened corruption or permanent data loss. This section covers systematic methods to identify data corruption, deleted data scenarios, and missing redo/InnoDB log issues.

1.1 Detecting Data Corruption

Step 1: Check the MySQL/MariaDB Error Log for corruption indicators:

```
# Locate the error log path
$ mysql -u root -p -e "SHOW VARIABLES LIKE 'log_error';"

# Search for corruption-related messages
$ grep -iE "corrupt|crash|repair|damaged|mismatch" /var/log/mysqld.log
$ grep -iE "InnoDB: Database page corruption" /var/log/mysqld.log
$ tail -500 /var/log/mysqld.log | grep -i "error"
```

Look for messages containing: *"page corruption"*, *"checksum mismatch"*, *"index is corrupted"*, or *"table is marked as crashed"*.

Step 2: Run CHECK TABLE on suspected tables:

```
mysql> CHECK TABLE database_name.table_name EXTENDED;

# Check ALL tables in a specific database
mysql> SELECT CONCAT("CHECK TABLE ", TABLE_SCHEMA, ".", TABLE_NAME, "
EXTENDED;")
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'your_database';
```

Step 3: Use mysqlcheck to scan all databases at once:

```
$ mysqlcheck -u root -p --all-databases --check --extended
$ mysqlcheck -u root -p --check --extended your_database
```

Step 4: Verify InnoDB tablespace integrity:

```
mysql> SHOW ENGINE INNODB STATUS\G

# Check for zero-size .ibd files (strong corruption indicator)
$ find /apps/mysql/data/ -name "*.ibd" -size 0 -print

# Check disk health via SMART
$ smartctl -a /dev/sda | grep -E
"Reallocated|Current_Pending|Offline_Uncorrectable"

# Check dmesg for I/O errors
$ dmesg | grep -iE "error|fault|I/O" | tail -30
```

Tip: If CHECK TABLE returns "OK" but queries still fail, the corruption may be in secondary indexes. Try ANALYZE TABLE and OPTIMIZE TABLE as well.

1.2 Identifying Deleted Data via Binary Logs

Step 1: Verify binary logging is enabled and locate log files:

```
mysql> SHOW VARIABLES LIKE 'log_bin';
mysql> SHOW VARIABLES LIKE 'binlog_format';
mysql> SHOW VARIABLES LIKE 'log_bin_basename';

# List all available binary logs with sizes
mysql> SHOW BINARY LOGS;
mysql> SHOW MASTER STATUS;

# Verify binary log files on disk
$ ls -lhr /apps/mysql/binlog/mysql-bin.*
```

Step 2: Decode binary logs to find DELETE/DROP/TRUNCATE statements:

```
# Decode binary log to readable SQL
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000045 > /tmp/binlog_decoded.sql

# Search for destructive operations
$ grep -n -i "DELETE FROM\|DROP TABLE\|TRUNCATE\|DROP DATABASE" \
/tmp/binlog_decoded.sql

# Filter by specific time range
$ mysqlbinlog --start-datetime="2026-03-24 00:00:00" \
--stop-datetime="2026-03-25 00:00:00" \
--base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000045 | grep -i "delete"
```

Step 3: Identify exact binary log position of the destructive event:

```
# Decode with row-level detail to see exact affected rows
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose --verbose \
/apps/mysql/binlog/mysql-bin.000045 | less

# Look for "# at <POSITION>" markers around the event
# Note: position BEFORE the event = --stop-position
# Note: position AFTER the event = --start-position for resume

# Search by database name for faster filtering
$ mysqlbinlog --database=your_database \
--base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000045 | grep -B5 -A10 "DELETE"
```

Step 4: Check INFORMATION_SCHEMA for table existence:

```
mysql> SELECT TABLE_NAME, ENGINE, TABLE_ROWS, CREATE_TIME, UPDATE_TIME
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'your_database'
ORDER BY UPDATE_TIME DESC;

# Check for orphaned .ibd files (table dropped but file remains)
$ ls -la /apps/mysql/data/your_database/
```

1.3 Diagnosing Missing Redo Logs (InnoDB)

Step 1: Check current redo log configuration:

```
mysql> SHOW VARIABLES LIKE 'innodb_log_file_size';
mysql> SHOW VARIABLES LIKE 'innodb_log_files_in_group';
mysql> SHOW VARIABLES LIKE 'innodb_log_group_home_dir';

# MySQL 8.0.30+ uses innodb_redo_log_capacity instead
mysql> SHOW VARIABLES LIKE 'innodb_redo_log_capacity';
```

Step 2: Verify redo log files exist on disk:

```
# Check for ib_logfile0 and ib_logfile1
$ ls -lh /apps/mysql/data/ib_logfile*

# MySQL 8.0.30+ stores redo logs in #innodb_redo directory
$ ls -lh /apps/mysql/data/#innodb_redo/
```

Step 3: Check error log for redo log related errors:

```
$ grep -iE "redo log|ib_logfile|log file|recovery" /var/log/mysqld.log
```

```
# Common error patterns:  
# "InnoDB: Missing redo log file ib_logfile0"  
# "InnoDB: Redo log file size mismatch"  
# "InnoDB: Cannot continue operation"
```

Step 4: Check InnoDB recovery status:

```
mysql> SHOW ENGINE INNODB STATUS\G
```

```
# Focus on the LOG section:  
# Log sequence number -> current LSN  
# Log flushed up to -> flushed LSN  
# Pages flushed up to -> flushed pages LSN  
# Last checkpoint at -> checkpoint LSN
```

WARNING: If redo log files are missing and MySQL refuses to start, do NOT attempt to create empty redo log files manually. This will cause data loss. Use `innodb_force_recovery` instead (covered in Section 3).

Section 2: Precautionary Steps Before Remediation

Before executing any remediation, you must secure the environment to prevent further damage. Skipping these steps can turn a recoverable situation into permanent data loss.

2.1 Backup Current State (Even if Corrupted)

Step 1: Take a raw filesystem-level backup immediately:

```
# STOP MySQL first if possible
$ systemctl stop mysql

# Raw copy of the entire data directory
$ cp -a /apps/mysql/data /apps/mysql/backups/data_raw_$(date +%Y%m%d_%H%M%S)

# If disk space is limited, use tar with compression
$ tar czf /apps/mysql/backups/data_raw_$(date +%Y%m%d_%H%M%S).tar.gz
/apps/mysql/data/
```

Step 2: If MySQL is still running, take a logical backup:

```
# Backup all databases
$ mysqldump -u root -p --all-databases --single-transaction \
--routines --triggers --events > /apps/mysql/backups/full_dump_$(date
+%Y%m%d).sql

# Backup specific critical database
$ mysqldump -u root -p --single-transaction --routines \
your_database > /apps/mysql/backups/your_database_$(date +%Y%m%d).sql
```

Step 3: Backup binary logs (critical for PITR):

```
# Copy all binary logs
$ cp /apps/mysql/binlog/mysql-bin.* /apps/mysql/backups/binlogs/
$ cp /apps/mysql/binlog/mysql-bin.index /apps/mysql/backups/binlogs/
```

Step 4: Backup the InnoDB redo and undo logs:

```
$ cp /apps/mysql/data/ib_logfile* /apps/mysql/backups/redo_logs/
$ cp /apps/mysql/data/undo_* /apps/mysql/backups/undo_logs/
$ cp /apps/mysql/data/ibdata1 /apps/mysql/backups/
```

WARNING: NEVER skip the backup step. Even a corrupted backup is better than no backup. Some remediation steps are destructive and irreversible.

2.2 Environment Safeguards

Step 1: Prevent application writes during recovery:

```
mysql> SET GLOBAL read_only = ON;
mysql> SET GLOBAL super_read_only = ON;

# Block network access at firewall level
$ iptables -A INPUT -p tcp --dport 3306 -j DROP
$ iptables -A INPUT -p tcp --dport 3306 -s 127.0.0.1 -j ACCEPT
```

Step 2: Kill problematic sessions and document server state:

```
mysql> SHOW FULL PROCESSLIST;
mysql> SELECT CONCAT("KILL ", ID, ";") FROM information_schema.PROCESSLIST
WHERE USER != 'system user' AND COMMAND != 'Binlog Dump';

# Record current status for later comparison
$ mysql -u root -p -e "CHECKSUM TABLE db.table1, db.table2;" \
> /tmp/checksums_before.txt
```

2.3 Communication & Change Management

- Notify stakeholders — Application teams, management, and SRE/on-call.
- Open an incident ticket — Log timeline, symptoms, and actions taken.
- Establish a maintenance window — Get formal approval for downtime.
- Confirm rollback plan — Ensure you have a known-good backup to restore from.
- Pair with a second DBA — Four eyes on destructive operations.

Section 3: Remediation Steps

This section provides detailed recovery procedures. Always start with the least destructive approach and escalate only if needed.

3.1 Recovering Corrupted Tables

▪ Method A: REPAIR TABLE (MyISAM) / Online Rebuild (InnoDB)

Step 1: For MyISAM tables — run REPAIR TABLE:

```
mysql> REPAIR TABLE database_name.table_name EXTENDED;  
$ mysqlcheck -u root -p --repair --extended your_database
```

Step 2: For InnoDB tables — force a table rebuild:

```
mysql> ALTER TABLE database_name.table_name ENGINE=InnoDB;  
mysql> OPTIMIZE TABLE database_name.table_name;  
  
# If table cannot be read, dump and re-import:  
$ mysqldump -u root -p database_name table_name >  
/apps/mysql/backups/table_backup.sql  
mysql> DROP TABLE database_name.table_name;  
$ mysql -u root -p database_name < /apps/mysql/backups/table_backup.sql
```

▪ Method B: InnoDB Force Recovery (for severe corruption)

Step 3: Start MySQL with innodb_force_recovery (escalate levels 1-6):

```
# Edit MySQL configuration  
$ vi /etc/my.cnf  
  
# Add under [mysqld] section:  
# innodb_force_recovery = 1  
  
# Recovery levels:  
# 1 = SRV_FORCE_IGNORE_CORRUPT - Skip corrupt pages  
# 2 = SRV_FORCE_NO_BACKGROUND - Stop background ops  
# 3 = SRV_FORCE_NO_TRX_UNDO - Skip transaction rollback  
# 4 = SRV_FORCE_NO_IBUF_MERGE - Skip insert buffer merge  
# 5 = SRV_FORCE_NO_UNDO_LOG_SCAN - Skip undo log scan  
# 6 = SRV_FORCE_NO_LOG_REDO - Skip redo log apply
```

Step 4: Start with level 1, dump data, rebuild if needed:

```

$ systemctl start mysql

# Once MySQL starts, IMMEDIATELY dump all data:
$ mysqldump -u root -p --all-databases --routines \
--triggers --events > /apps/mysql/backups/emergency_full_dump.sql

# Then rebuild from scratch:
$ systemctl stop mysql
# Remove innodb_force_recovery from /etc/my.cnf !
$ mv /apps/mysql/data /apps/mysql/data_old
$ mysqld --initialize-insecure --user=mysql \
--datadir=/apps/mysql/data
$ systemctl start mysql
$ mysql -u root < /apps/mysql/backups/emergency_full_dump.sql

```

WARNING: At recovery level 4 and above, you CANNOT write to InnoDB tables. Only SELECT and mysqldump will work. Dump your data first, then rebuild.

3.2 Recovering Deleted Data — mysqlbinlog Scenarios

The mysqlbinlog utility is the primary tool for Point-in-Time Recovery (PITR). Below are seven real-world scenarios covering different types of accidental data loss and the exact commands to recover from each.

Pre-requisite for all scenarios: Binary logging must be enabled (`log_bin = ON`) and you need a recent full backup taken BEFORE the destructive event. The backup's binary log position is your starting point.

Scenario 1: Recover Accidentally Deleted Rows (DELETE)

A developer accidentally ran `DELETE FROM orders WHERE status = 'pending'` without a proper WHERE clause or with too broad a filter, wiping thousands of rows.

Step 1: Find the exact binary log position of the DELETE event:

```
# Decode the binlog around the suspected time window
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose --verbose \
--start-datetime="2026-03-24 14:00:00" \
--stop-datetime="2026-03-24 14:30:00" \
/apps/mysql/binlog/mysql-bin.000052 > /tmp/binlog_delete_analysis.sql

# Search for the DELETE event
$ grep -n -B5 "DELETE FROM.*orders" /tmp/binlog_delete_analysis.sql

# Example output:
# 1842: # at 48750 <-- position BEFORE the DELETE
# 1845: ### DELETE FROM `production`.`orders`
# 1860: # at 52340 <-- position AFTER the DELETE
```

Step 2: Restore the last full backup to a recovery instance:

```
# On a SEPARATE recovery server (NEVER on production)
$ mysql -u root -p -e "CREATE DATABASE recovery_db;"
$ mysql -u root -p recovery_db < /apps/mysql/backups/last_full_backup.sql
```

Step 3: Replay binary logs UP TO the position before the DELETE:

```
# Replay from backup position to just BEFORE the DELETE
# (backup recorded at binlog position 12345)
$ mysqlbinlog --start-position=12345 \
--stop-position=48750 \
--database=production \
/apps/mysql/binlog/mysql-bin.000050 \
/apps/mysql/binlog/mysql-bin.000051 \
/apps/mysql/binlog/mysql-bin.000052 \
| mysql -u root -p recovery_db
```

Step 4: SKIP the bad DELETE and replay everything AFTER it:

```
# Resume replay AFTER the DELETE event
$ mysqlbinlog --start-position=52340 \
/apps/mysql/binlog/mysql-bin.000052 \
/apps/mysql/binlog/mysql-bin.000053 \
| mysql -u root -p recovery_db
```

Step 5: Copy recovered rows back to production:

```
# Export recovered table
$ mysqldump -u root -p recovery_db orders >
/apps/mysql/backups/orders_recovered.sql

# Selectively insert only the missing rows into production
mysql> INSERT IGNORE INTO production.orders
SELECT * FROM recovery_db.orders
WHERE id NOT IN (SELECT id FROM production.orders);

# Verify row count
mysql> SELECT COUNT(*) FROM production.orders;
```

Scenario 2: Recover a Dropped Table (DROP TABLE)

An operator accidentally ran **DROP TABLE customers** on the production database, completely removing the table and its data.

Step 1: Locate the DROP TABLE event in binary logs:

```
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000055 \
| grep -n -B3 "DROP TABLE"

# Note the "# at <position>" BEFORE the DROP event
```

Step 2: Restore backup and replay binlogs excluding the DROP:

```
# Restore full backup to recovery server
$ mysql -u root -p recovery_db < /apps/mysql/backups/last_full_backup.sql

# Replay binlogs, stopping just BEFORE the DROP
$ mysqlbinlog --start-position=12345 \
--stop-position=78900 \
--database=production \
/apps/mysql/binlog/mysql-bin.000053 \
/apps/mysql/binlog/mysql-bin.000054 \
/apps/mysql/binlog/mysql-bin.000055 \
| mysql -u root -p recovery_db
```

Step 3: Export the recovered table and recreate on production:

```
# Dump the full table with schema + data
$ mysqldump -u root -p recovery_db customers >
/apps/mysql/backups/customers_full.sql

# Import to production
$ mysql -u root -p production < {P_BACKUP}/customers_full.sql

# Resume remaining binlog events AFTER the DROP position
$ mysqlbinlog --start-position=78950 \
--database=production \
/apps/mysql/binlog/mysql-bin.000055 \
/apps/mysql/binlog/mysql-bin.000056 \
| mysql -u root -p production
```

Note: When replaying binlogs after restoring the table, ensure the replay does not include any additional references to the dropped table that could fail. Use --database filter to isolate events.

Scenario 3: Recover from TRUNCATE TABLE

TRUNCATE TABLE is a DDL operation (not row-level DELETE), so it logs differently in binary logs. The table structure remains but all data is removed instantly.

Step 1: Find the TRUNCATE event in the binlog:

```
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000060 \
| grep -n -B3 "TRUNCATE"

# Example output:
# 4521: # at 156780 <-- position BEFORE TRUNCATE
# 4523: TRUNCATE TABLE `production`.`audit_logs`
# 4525: # at 156850 <-- position AFTER TRUNCATE
```

Step 2: Restore and replay up to just before the TRUNCATE:

```
$ mysql -u root -p recovery_db < /apps/mysql/backups/last_full_backup.sql

$ mysqlbinlog --start-position=12345 \
--stop-position=156780 \
--database=production \
/apps/mysql/binlog/mysql-bin.000058 \
/apps/mysql/binlog/mysql-bin.000059 \
/apps/mysql/binlog/mysql-bin.000060 \
| mysql -u root -p recovery_db
```

Step 3: Extract data and reload into production table:

```
# Dump ONLY the data (table structure already exists in prod)
$ mysqldump -u root -p --no-create-info \
recovery_db audit_logs > /apps/mysql/backups/audit_logs_data.sql

# Reload data into the (now empty) production table
$ mysql -u root -p production < /apps/mysql/backups/audit_logs_data.sql

# Resume binlog replay AFTER the TRUNCATE
$ mysqlbinlog --start-position=156850 \
/applications/mysql/binlog/mysql-bin.000060 \
| mysql -u root -p production
```

WARNING: TRUNCATE resets AUTO_INCREMENT. After restoring data, verify that the AUTO_INCREMENT counter is correct: ALTER TABLE audit_logs AUTO_INCREMENT = <next_value>;

Scenario 4: Recover from Accidental UPDATE (Wrong WHERE Clause)

Someone ran `UPDATE users SET email = 'test@test.com'` without a WHERE clause, overwriting the email column for every row in the table.

Step 1: Find the UPDATE event with double-verbose to see old values:

```
# Double --verbose shows BOTH old and new row values
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose --verbose \
--start-datetime="2026-03-24 09:00:00" \
--stop-datetime="2026-03-24 10:00:00" \
/applications/mysql/binlog/mysql-bin.000048 > /tmp/update_analysis.sql

# Search for the UPDATE
$ grep -n -A20 "UPDATE.*users" /tmp/update_analysis.sql | head -60

# Output shows:
# ### UPDATE `production`.`users`
# ### WHERE <-- original row values (BEFORE)
# ### @1=1 <-- column 1 old value
# ### @2='real@email.com' <-- column 2 old value
# ### SET <-- new row values (AFTER)
# ### @1=1
# ### @2='test@test.com'
```

Step 2: Extract the original values from binlog and generate reversal SQL:

```
# Use mysqlbinlog to extract the UPDATE events as SQL
# Then reverse them: swap WHERE and SET clauses

# Method: Restore backup + replay to point before UPDATE
$ mysql -u root -p recovery_db < /apps/mysql/backups/last_full_backup.sql

$ mysqlbinlog --start-position=12345 \
  --stop-position=95000 \
  --database=production \
  /apps/mysql/binlog/mysql-bin.000046 \
  /apps/mysql/binlog/mysql-bin.000047 \
  /apps/mysql/binlog/mysql-bin.000048 \
  | mysql -u root -p recovery_db
```

Step 3: Generate UPDATE statements to fix production from recovery data:

```
# Generate corrective UPDATE statements
mysql> SELECT CONCAT('UPDATE production.users SET email = \'',
  r.email, '\ ' WHERE id = ', r.id, ';')
FROM recovery_db.users r
INNER JOIN production.users p ON r.id = p.id
WHERE r.email != p.email;

# Save and execute the corrective SQL
$ mysql -u root -p -e "...above query..." > /tmp/fix_emails.sql
$ mysql -u root -p production < /tmp/fix_emails.sql
```

Pro Tip: Tools like binlog2sql (by Danfengcao) can automatically generate reversal/flashback SQL from binary logs, swapping INSERT/DELETE and reversing UPDATE SET/WHERE — saving significant manual effort.

Scenario 5: PITR Skipping a Bad Transaction Using GTIDs

When GTID-based replication is enabled, you can skip a specific transaction identified by its GTID without needing to find exact byte positions. This is the cleanest approach for environments using GTIDs.

Step 1: Identify the GTID of the bad transaction:

```
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose \
  /apps/mysql/binlog/mysql-bin.000055 \
  | grep -B2 -A10 "DELETE FROM.*orders"

# Look for the SET @@SESSION.GTID_NEXT line:
# SET @@SESSION.GTID_NEXT= 'a1b2c3d4-...:12345'
# The GTID is: a1b2c3d4-...:12345
```

Step 2: Restore backup and replay binlogs, excluding that GTID:

```
# Restore the base backup
$ mysql -u root -p recovery_db < /apps/mysql/backups/last_full_backup.sql

# Replay ALL binlogs, but EXCLUDE the bad transaction
$ mysqlbinlog --exclude-gtids="a1b2c3d4-xxxx-xxxx-xxxx:12345" \
--database=production \
/apps/mysql/binlog/mysql-bin.000050 \
/apps/mysql/binlog/mysql-bin.000051 \
/apps/mysql/binlog/mysql-bin.000052 \
/apps/mysql/binlog/mysql-bin.000053 \
/apps/mysql/binlog/mysql-bin.000054 \
/apps/mysql/binlog/mysql-bin.000055 \
| mysql -u root -p recovery_db
```

Step 3: Skip a RANGE of bad GTIDs (multiple bad transactions):

```
# Exclude multiple GTIDs or a GTID range
$ mysqlbinlog \
--exclude-gtids="a1b2c3d4-xxxx:12345,a1b2c3d4-xxxx:12350-12355" \
/apps/mysql/binlog/mysql-bin.000050 \
/apps/mysql/binlog/mysql-bin.000051 \
| mysql -u root -p recovery_db

# Alternatively, replay ONLY specific GTIDs
$ mysqlbinlog \
--include-gtids="a1b2c3d4-xxxx:1-12344,a1b2c3d4-xxxx:12346-99999" \
/apps/mysql/binlog/mysql-bin.000050 \
| mysql -u root -p recovery_db
```

GTID Advantage: With GTIDs, you do not need to calculate byte-level positions across multiple binlog files. A single `--exclude-gtids` flag cleanly skips the exact transaction regardless of which binlog file it lives in.

Scenario 6: Recover from DROP DATABASE

The most catastrophic accidental event: **DROP DATABASE production** was executed. The entire database, all tables, views, routines, and events are gone.

DANGER: DROP DATABASE is IRREVERSIBLE without binary logs and a backup. If neither exists, the data is permanently lost. Act fast — do not restart MySQL as the .ibd pages may still be recoverable from disk using forensic tools.

Step 1: Locate the DROP DATABASE event:


```
$ mysqlbinlog --base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000070 \
| grep -n -B5 "DROP DATABASE"

# Note the position number BEFORE the DROP DATABASE line
```

Step 2: Full restore: backup + binlog replay up to the DROP:

```
# Restore the full backup to a recovery server
$ mysql -u root -p < /apps/mysql/backups/last_full_backup.sql

# Replay ALL binlogs, stopping BEFORE DROP DATABASE
$ mysqlbinlog --start-position=12345 \
--stop-position=245000 \
/apps/mysql/binlog/mysql-bin.000065 \
/apps/mysql/binlog/mysql-bin.000066 \
/apps/mysql/binlog/mysql-bin.000067 \
/apps/mysql/binlog/mysql-bin.000068 \
/apps/mysql/binlog/mysql-bin.000069 \
/apps/mysql/binlog/mysql-bin.000070 \
| mysql -u root -p
```

Step 3: Skip the DROP DATABASE and replay remaining events:

```
# Resume AFTER the DROP DATABASE event
$ mysqlbinlog --start-position=245100 \
/apps/mysql/binlog/mysql-bin.000070 \
/apps/mysql/binlog/mysql-bin.000071 \
| mysql -u root -p

# Verify the database is fully restored
mysql> SHOW DATABASES;
mysql> USE production;
mysql> SHOW TABLES;
mysql> SELECT COUNT(*) AS total_tables FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'production';
```

Scenario 7: Selective Table Recovery from Full Binlog Stream

You need to recover only ONE specific table from the binary log stream without affecting other tables. This is useful when a single table was corrupted or deleted but the rest of the database is fine.

Step 1: Filter binlog events for a specific database only:

```
# The --database flag filters events by default database
$ mysqlbinlog --database=production \
--base64-output=DECODE-ROWS --verbose \
/apps/mysql/binlog/mysql-bin.000045 > /tmp/prod_only_events.sql
```

Step 2: Restore only the affected table from backup:

```
# Restore ONLY the single table from the full backup
# First, create a temp database for recovery
mysql> CREATE DATABASE temp_recovery;

# Extract the single table from the dump file
$ sed -n "/^-- Table structure for table .payments/,/^-- Table structure for
table/p" \
/apps/mysql/backups/last_full_backup.sql > /tmp/payments_only.sql

# OR use mysqldump --tables for a cleaner approach
# (if backup server is accessible)
$ mysqldump -u root -p production payments \
--single-transaction > /tmp/payments_only.sql
```

Step 3: Replay binlog events for only that table using rewrite-db:

```
# Replay events for the production DB into temp_recovery,
# then manually extract only the payments table changes
$ mysqlbinlog --database=production \
--start-position=12345 \
--stop-position=48750 \
/apps/mysql/binlog/mysql-bin.000050 \
/apps/mysql/binlog/mysql-bin.000051 \
| grep -v "^USE " \
| mysql -u root -p temp_recovery

# Alternatively, use --rewrite-db to redirect to temp DB
$ mysqlbinlog --database=production \
--rewrite-db="production->temp_recovery" \
--start-position=12345 \
--stop-position=48750 \
/apps/mysql/binlog/mysql-bin.000050 \
| mysql -u root -p
```

Step 4: Copy the recovered table back to production:

```
# Rename/swap the table atomically
mysql> RENAME TABLE production.payments TO production.payments_damaged,
temp_recovery.payments TO production.payments;

# Verify
mysql> SELECT COUNT(*) FROM production.payments;
mysql> CHECK TABLE production.payments EXTENDED;

# Cleanup
mysql> DROP TABLE production.payments_damaged;
mysql> DROP DATABASE temp_recovery;
```

WARNING: The `--database` filter only matches events whose default database matches. Cross-database statements like `USE db1; DELETE FROM db2.table` may be missed. Always verify with `--verbose` output.

3.3 Recovery via Mariabackup / Xtrabackup

Step 1: Prepare and restore from physical backup:

```
# Prepare the base backup
$ mariabackup --prepare --target-dir=/apps/mysql/backups/base/

# Apply incremental backups (if available)
$ mariabackup --prepare --target-dir=/apps/mysql/backups/base/ \
--incremental-dir=/apps/mysql/backups/inc1/

# Stop MySQL and restore
$ systemctl stop mysql
$ mariabackup --copy-back --target-dir=/apps/mysql/backups/base/
$ chown -R mysql:mysql /apps/mysql/data
$ systemctl start mysql
```

Step 2: Apply remaining binlogs after backup for full PITR:

```
# Check the binlog position recorded by the backup
$ cat /apps/mysql/backups/base/xtrabackup_binlog_info
# Output: mysql-bin.000045 12345

# Replay binlogs from that position, skipping the bad event
$ mysqlbinlog --start-position=12345 \
--stop-position=<bad_event_pos> \
/apps/mysql/binlog/mysql-bin.000045 \
| mysql -u root -p
```

Percona XtraBackup users: Replace mariabackup with xtrabackup. The syntax is nearly identical.

3.4 Rebuilding Missing / Corrupted Redo Logs

Step 1: Attempt a clean restart first (MySQL may auto-recover):

```
$ systemctl start mysql
$ systemctl status mysql
$ tail -50 /var/log/mysqld.log | grep -i "redo\|recovery\|ready"
```

Step 2: If MySQL fails to start, use `innodb_force_recovery = 6`:

```
$ cat >> /etc/my.cnf << EOF
[mysqld]
innodb_force_recovery = 6
EOF

$ systemctl start mysql

# IMMEDIATELY dump everything
$ mysqldump -u root -p --all-databases --routines \
--triggers --events > /apps/mysql/backups/redo_recovery_dump.sql
```

Step 3: Clean rebuild with new redo logs:

```
$ systemctl stop mysql
# Remove force_recovery from /etc/my.cnf!
$ vi /etc/my.cnf

# Remove old redo log files
$ rm /apps/mysql/data/ib_logfile*
# MySQL 8.0.30+: rm -rf /apps/mysql/data/#innodb_redo/*

# Start MySQL – it will recreate redo logs automatically
$ systemctl start mysql
$ ls -lh /apps/mysql/data/ib_logfile*
```

Step 4: If clean restart fails, full re-initialization:

```
$ systemctl stop mysql
$ mv /apps/mysql/data /apps/mysql/data_broken_$(date +%Y%m%d)
$ mysqld --initialize-insecure --user=mysql --datadir=/apps/mysql/data
$ systemctl start mysql
$ mysql -u root -e "ALTER USER 'root'@'localhost' IDENTIFIED BY 'NewPass!';"
$ mysql -u root -p < /apps/mysql/backups/redo_recovery_dump.sql
```

Step 5: Tune redo log size to prevent future issues:

```
# Recommended: 1-2x your peak hourly write volume
# In /etc/my.cnf under [mysqld]:

# MySQL 5.7 / 8.0 (pre-8.0.30)
# innodb_log_file_size = 512M
# innodb_log_files_in_group = 2

# MySQL 8.0.30+
# innodb_redo_log_capacity = 1G

# MariaDB 10.5+
# innodb_log_file_size = 512M
```

WARNING: Never delete redo logs while MySQL is running. Always stop the service first. MySQL will recreate them on next startup with the configured size.

Section 4: Post-Remediation Confirmation

After remediation is complete, systematically verify that the database is healthy, data is intact, and the system is performing as expected before handing back to applications.

4.1 Data Integrity Verification

Step 1: Run CHECK TABLE on all recovered tables:

```
$ mysqlcheck -u root -p --all-databases --check --extended
# Every table should return: OK
```

Step 2: Verify row counts match expected values:

```
mysql> SELECT TABLE_NAME, TABLE_ROWS
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'your_database'
ORDER BY TABLE_NAME;

# For exact count (TABLE_ROWS is approximate for InnoDB)
mysql> SELECT COUNT(*) FROM your_database.your_table;
```

Step 3: Compare table checksums against known-good values:

```
$ mysql -u root -p -e "CHECKSUM TABLE db.table1, db.table2;" \
> /tmp/checksums_after.txt
$ diff /tmp/checksums_before.txt /tmp/checksums_after.txt
```

Step 4: Validate foreign key relationships:

```
mysql> SET GLOBAL foreign_key_checks = ON;

# Find orphaned child records
mysql> SELECT c.id FROM child_table c
LEFT JOIN parent_table p ON c.parent_id = p.id
WHERE p.id IS NULL;

mysql> SELECT TABLE_NAME, AUTO_INCREMENT
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'your_database' AND AUTO_INCREMENT IS NOT NULL;
```

4.2 Performance & Stability Checks

Step 1: Verify InnoDB engine status is clean:

```
mysql> SHOW ENGINE INNODB STATUS\G
```

```
# Verify: Buffer pool hit rate > 99%  
# No pending reads/writes backlog  
# Log sequence numbers advancing normally
```

Step 2: Check redo logs and binlogs are functioning:

```
$ ls -lh /apps/mysql/data/ib_logfile*
```

```
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_os_log%';  
mysql> SHOW GLOBAL STATUS LIKE 'Innodb_log_waits';  
# Innodb_log_waits should be 0 or very low
```

```
# Verify binary logging is active  
mysql> SHOW MASTER STATUS;  
mysql> SHOW BINARY LOGS;
```

Step 3: Update index statistics and verify queries:

```
$ mysqlcheck -u root -p --analyze your_database  
mysql> EXPLAIN SELECT * FROM your_table WHERE indexed_column = 'value';
```

Step 4: Monitor for 30–60 minutes before declaring recovery complete:

```
$ tail -f /var/log/mysqld.log &  
  
$ mysqladmin -u root -p extended-status -i 10 | \  
grep -E "Threads_running|Questions|Slow_queries|Innodb_row_lock"  
  
# Check replication lag (if applicable)  
mysql> SHOW SLAVE STATUS\G  
# Seconds_Behind_Master should be 0
```

4.3 Final Sign-Off Checklist

- All CHECK TABLE commands return OK
- Row counts match expected / pre-incident values
- Table checksums are consistent
- InnoDB status shows no errors or warnings
- Redo log files exist and are properly sized
- Binary logging is enabled and active (SHOW MASTER STATUS)
- Application connectivity verified — reads and writes successful
- Replication is healthy with zero lag (if applicable)
- Monitoring alerts cleared / resolved

A fresh full backup has been taken post-recovery
Incident ticket updated with root cause and resolution
read_only and super_read_only disabled on production
Firewall rules reverted to normal
Stakeholders notified of recovery completion

RECOVERY COMPLETE — Database integrity confirmed. All systems nominal. Document findings in the post-incident review.

This runbook is a living document. Update it after every incident to reflect lessons learned. Maintain tested backups and rehearse recovery procedures quarterly.